

JUGNU — COMPLETE BACKEND ARCHITECTURE

Master architecture + implementation checklist

1. Core Architecture

```
React / Next.js
↓ REST API
Node.js + Express + TypeScript
↓
Prisma ORM → PostgreSQL
↑
Python AI / ML Service
```

Offline: Frontend → IndexedDB → Event Queue → Sync API → Backend → PostgreSQL

2. Foundation & Project Structure

• Node.js • TypeScript • Express • REST API • Prisma ORM • PostgreSQL • Python AI/ML • Docker

```
jugnu-backend/
├── src/
│   ├── config/
│   ├── middleware/
│   ├── modules/
│   │   ├── auth, users, patients, games,
│   │   ├── sessions, personalization,
│   │   ├── reminders, insights, alerts,
│   │   ├── family, health-workers
│   ├── services/
│   │   ├── AI, sync, notifications, encryption
│   ├── utils/
│   │   ├── app.ts / server.ts
│   ├── prisma/ (schema, seed, migrations)
│   ├── tests/
│   └── Dockerfile / docker-compose.yml / .env / package.json
```

3. Authentication & User Management

- Register, login, logout, current-user retrieval
- Password hashing + secure authentication
- Protected routes + role-based access
- Roles: Family Caregiver, Health Worker, Connected Family

4. Patient Management

- Create/read/update/delete patient profiles
- Basic demographics, language, caregiver and relevant location
- Consent status + personalization level

5. Consent & Privacy

- Consent for activities, personalization, voice, photos/personal content and family sharing
- Authentication, authorization, data minimization, encryption and secure secrets

6. Games / Cognitive Activities

- Generic: Object Match, Routine Sequencing, Pattern Recall
- Personalized: Who's Calling?, Remember When, My Daily Routine
- Game metadata: cognitive domain, difficulty, language, content, active status

7. Personalization

- Level 1: Generic — zero setup
- Level 2: Full Personalisation — memories, routines, caregiver voice, photos where enabled
- Optional: app works without personalization

8. Voice & Language

- Initially 2–3 NER languages
- Native-speaker audio + voice guidance
- Language selection + content/audio packs
- Expandable language-pack architecture

9. Session Management

- Start/end sessions, record attempts, retrieve history
- Flow: Start → Activity → Attempt → Metrics → Adaptive Engine → Next Activity → End

10. Performance / Attempt Data

- Accuracy, response time, errors, completion
- Game/session/difficulty/timestamp
- Raw input for adaptive intelligence

11. AI / Adaptive Engine

- Ability estimation per cognitive domain
- Adaptive difficulty targeting ~70–80% success
- Personal baseline from initial sessions
- Recent-vs-baseline longitudinal trend analysis
- Output: ability score, next difficulty, recommended activity
- Sustained significant change → Suggested Clinical Review; never diagnosis

12. Frustration Management

- Rule-based logic
- Avoid two consecutive failures where possible
- Reduce difficulty after repeated failures
- Gentler activity; session does not end because of failure

13. Caregiver Mood Check-in

- 3-second ■ / ■ / ■ check-in after sessions
- Separate caregiver trend
- 3+ consecutive low moods → caregiver-support alert
- Not a medical or mental-health diagnosis

14. Insights & Alerts

- Insights: performance trends, ability, adherence, longitudinal trends
- Alerts: Suggested Clinical Review, Caregiver Support, Reminders, System
- Use neutral wording such as “No flag raised”

15. Reminders & Notifications

- Medication, hydration, activity, appointments
- Notification service for reminders and care alerts

16. Caregiver Dashboard APIs

- Patient summary, session history, performance trends, activity adherence
- Reminders, alerts, care tips/resources

17. Health Worker System

- Assigned patient list + priority list
- Distributed: village → patients → priority → weekly visit plan
- Centralized: facility → room/ward → priority roster

18. Connected Family

- Primary caregiver: full access
- Connected family: view-only + weekly digest + voice clips
- Invite via email, mobile/OTP or invite code
- Connection + permissions + member removal

19. Offline-First & Sync

- IndexedDB for local gameplay/data
- Store events locally when offline
- Sync on reconnect
- Unique event IDs + idempotency + duplicate prevention
- Retries, conflict handling, sync status and last-sync time

20. Database Model

- User, FamilyMember, HealthWorker, Patient, Game, Session, Attempt
- Personalization, Reminder, Insight, Alert, Consent, FamilyConnection, MoodCheckIn, SyncEvent

21. Security

- Auth middleware, role/permission checks, validation/sanitization
- Secure tokens/sessions, password hashing, CORS
- Rate limiting where appropriate, safe errors, audit/logging

22. Error Handling & API Quality

- Central error middleware
- Consistent success/error response format
- Never expose internal database/server errors
- Validate before controller/service execution

23. Backend ↔ AI Communication

- Node/Express calls Python AI service
- Inputs: patient/domain/performance/history
- Outputs: ability score, next difficulty, recommended activity

24. Testing

- Unit: auth, patient, session, adaptive logic, alerts, sync, permissions
- Integration: API → Prisma → PostgreSQL
- E2E: Login → Patient → Session → Attempt → AI → Dashboard

25. Docker / Deployment

- Dockerize backend/supporting services
- Production PostgreSQL may be managed/cloud-hosted
- CI/CD: GitHub → tests → build → Docker → deploy

26. Environment & Secrets

- DATABASE_URL, JWT/session secret, AI_SERVICE_URL
- API_BASE_URL, FRONTEND_URL, NODE_ENV, PORT
- Never hard-code secrets

27. Logging & Monitoring

- Application/API logs
- Authentication events, database errors, sync failures
- Performance monitoring + structured logging

28. API Checklist

Area	Endpoints
Auth	POST /auth/register • POST /auth/login • POST /auth/logout • GET /auth/me
Patients	POST /patients • GET /patients • GET/PATCH/DELETE /patients/:id
Games	GET /games • GET /games/:id • GET /games/recommended/:patientId
Sessions	POST /sessions • POST /sessions/:id/attempts • POST /sessions/:id/end • GET /sessions/:id • GET /patients/:id/sessions
Personalization	GET/POST/PATCH /patients/:id/personalization
Reminders	GET/POST /patients/:id/reminders • PATCH/DELETE /reminders/:id
Insights	GET /patients/:id/insights • GET /patients/:id/trends • GET /patients/:id/ability
Alerts	GET /patients/:id/alerts • PATCH /alerts/:id
Family	POST /family/invite • GET /family/members/:patientId • DELETE /family/member/:id
Health Worker	GET /health-workers/patients • GET /health-workers/priority-list • GET /health-workers/visit-plan
Sync	POST /sync

29. Backend Team Division

Backend Guy 1 — Core Backend

- Node + Express
- Prisma + PostgreSQL
- Auth + roles
- Patients + consent
- Games + sessions + attempts
- Personalization
- Offline sync
- Security

Backend Guy 2 — Intelligence + Care

- Python AI/ML
- Ability estimation + adaptive difficulty
- Personal baseline + trend analysis
- Insights + alerts
- Caregiver dashboard
- Health-worker APIs
- Reminders
- Connected family + notifications

Jointly freeze before parallel development

- Database schema
- API request/response contracts
- Authentication method
- AI input/output format

30. MASTER COMPLETION CHECKLIST

Backend Area	Status
Node + Express setup	■
TypeScript	■
PostgreSQL	■
Prisma	■

Backend Area	Status
Database schema	■
Migrations	■
Auth	■
Roles / Permissions	■
Patients	■
Consent	■
Games	■
Sessions	■
Attempts / Performance	■
Personalization	■
Language / Voice assets	■
AI Ability Estimation	■
Adaptive Difficulty	■
Personal Baseline	■
Trend Analysis	■
Suggested Clinical Review	■
Caregiver Dashboard APIs	■
Health Worker APIs	■
Reminders	■
Caregiver Mood	■
Connected Family	■
Notifications	■
Offline Sync	■
Security	■
Tests	■
Docker	■
CI/CD	■
Deployment	■
Logging / Monitoring	■